🎓 **Exam Type: Practical**
⏳ **Duration: 6 Hours**

"House Price Prediction — Regression with Supervised Learning"

Project Overview

You are working as a Junior Data Scientist at a PropTech startup similar to NoBroker or MagicBricks. The product team wants an automated house price estimator to help buyers and sellers in the Indian market benchmark property values.

You are given the Ames Housing Dataset (a popular regression benchmark from Kaggle / UCI), which contains 79 features describing almost every aspect of residential homes. Your task is to build a complete end-to-end Supervised Learning pipeline that predicts the sale price of a house (regression task).

📁 **Dataset: Ames Housing Dataset (House Prices: Advanced Regression Techniques)**

🔗 Kaggle Link:
https://www.kaggle.com/competitions/house-prices-advanced-regression-techniques/data

📊 Rows: 1,460 training records | 79 Features (numeric + categorical)

🎯 Target Column: SalePrice (continuous — USD sale price of the house)

⚠️ Note: Use only the train.csv file for this exam (perform your own train-test split).

🎯 Learning Objectives

By completing this exam, students will demonstrate:

- Understanding and framing a real-world regression problem.
- Proficiency in EDA for continuous target variables (distribution, skewness, outliers).
- Handling mixed data types: high-cardinality categoricals, ordinal features, and numerical skew.
- Building and evaluating multiple regression models (Linear, Ridge, Lasso, Random Forest, XGBoost).
- Model evaluation with RMSE, MAE, R^2, and cross-validation.
- Residual analysis and interpretation for regression diagnostics.
- Packaging a clean sklearn Pipeline and submitting via GitHub.

🧠 Step 1: Problem Framing & Theory Notes

Before writing any code, write the following as Markdown cells in your notebook:

1. What is Regression? How is it different from Classification? Give 2 real-world Indian examples (e.g., predicting EMI at HDFC, predicting crop yield in Maharashtra).

2. What is the difference between Simple Linear Regression, Multiple Linear Regression, and Regularised Regression (Ridge / Lasso)?
3. Explain Overfitting and Underfitting. How does regularisation (L1/L2) help prevent them?
4. Define RMSE, MAE, and R^2. When would you prefer RMSE over MAE?
5. What is Cross-Validation (k-Fold)? Why is it preferred over a single train-test split for model evaluation?

💡 Tip: Keep all theory cells concise — 2 to 4 bullet points per question. Clarity counts more than length.

📂 Step 2: Dataset Loading & Exploratory Data Analysis

2.1 Load & Understand the Dataset

- Load train.csv using pandas. Print .shape, .info(), and .describe().
- Identify: (a) how many numerical columns, (b) how many categorical columns, (c) how many columns have missing values.
- Plot a histogram and Q-Q plot of the target variable SalePrice. Is it normally distributed? Apply log1p transformation and re-plot.

2.2 Univariate Analysis

- Plot histograms for all numerical features. Identify the top 5 most skewed columns.
- Plot countplots for high-impact categorical columns: Neighborhood, BldgType, HouseStyle.

2.3 Bivariate Analysis

- Plot a heatmap of the top 15 features most correlated with SalePrice.
- Scatter plots: GrLivArea vs SalePrice, TotalBsmtSF vs SalePrice, GarageArea vs SalePrice.
- Boxplots: OverallQual vs SalePrice, Neighborhood vs SalePrice (which neighborhood is the priciest?).
- Identify and flag any outliers visible in GrLivArea vs SalePrice (hint: two extreme points exist).

📌 Key features to focus on: GrLivArea, TotalBsmtSF, OverallQual, GarageArea, YearBuilt, Neighborhood, BldgType, SalePrice

🔧 Step 3: Data Preprocessing & Feature Engineering

3.1 Handle Missing Values

- Report all columns with missing values and their missing %.
- For categorical columns where NaN means 'None' (e.g., PoolQC, Fence, GarageType) — fill with 'None'.
- For numerical columns where NaN means 0 (e.g., GarageArea, BsmtFinSF1) — fill with 0.
- For remaining numerical columns — fill with median.
- Drop columns with >80% missing values.

3.2 Outlier Removal

- Remove the 2 outlier rows in GrLivArea where GrLivArea > 4000 and SalePrice < 300,000.
- Briefly explain why keeping these outliers would harm a linear model.

3.3 Feature Engineering

- Create TotalSF = TotalBsmtSF + 1stFlrSF + 2ndFlrSF (total usable area).
- Create HouseAge = YrSold - YearBuilt.
- Create RemodAge = YrSold - YearRemodAdd.
- Create HasGarage = 1 if GarageArea > 0 else 0.
- Create HasPool = 1 if PoolArea > 0 else 0.

3.4 Encoding & Scaling

- Apply Ordinal Encoding to quality/condition columns: ExterQual, KitchenQual, BsmtQual, FireplaceQu (map: Po→1, Fa→2, TA→3, Gd→4, Ex→5, None→0).
- Apply One-Hot Encoding to nominal categoricals with ≤10 unique values (e.g., BldgType, HouseStyle, SaleCondition).
- Drop or label-encode high-cardinality columns (Neighborhood: use mean-target encoding or label encode).
- Apply log1p transformation to skewed numerical features (skewness > 0.75).
- Apply StandardScaler to all continuous numerical features.
- Use ColumnTransformer + Pipeline to chain all transformations cleanly.

3.5 Train-Test Split

- Apply log1p to y (SalePrice) before splitting — this is your target.
- Split: 80% train / 20% test using random_state=42.
- Print shapes of X_train, X_test, y_train, y_test.

📐 Step 4: Regression Models — Linear, Ridge & Lasso

4.1 Linear Regression (Baseline)

- Train a LinearRegression model. Predict on X_test.
- Report: RMSE, MAE, R² on test set (remember to expm1 your predictions before computing metrics on original scale).
- Plot: Actual vs Predicted scatter plot. A perfect model falls on the diagonal — how close are you?

4.2 Ridge Regression (L2)

- Train RidgeCV(alphas=[0.01, 0.1, 1, 10, 100]) — let it auto-select the best alpha.
- Print the best alpha selected.
- Report: RMSE, MAE, R² on test set.
- Plot coefficients: top 20 most impactful features by absolute coefficient value.

4.3 Lasso Regression (L1)

- Train LassoCV(alphas=[0.0001, 0.001, 0.01, 0.1, 1], max_iter=10000) — let it auto-select best alpha.
- Print the best alpha and how many features were zeroed out (coefficient = 0).
- Report: RMSE, MAE, R² on test set.
- Compare: which model (Ridge vs Lasso) performed better and why?

⚠️ Remember: All metrics (RMSE, MAE) must be computed after applying expm1() to reverse the log1p target transformation, so they are in original USD units.

🌳 Step 5: Advanced Models — Random Forest & XGBoost Regressor

5.1 Random Forest Regressor

- Train RandomForestRegressor(n_estimators=200, max_depth=10, random_state=42, n_jobs=-1).
- Report: RMSE, MAE, R² on test set.
- Plot feature importances — top 15 features.

5.2 XGBoost Regressor

- Train XGBRegressor(n_estimators=500, learning_rate=0.05, max_depth=4, subsample=0.8, colsample_bytree=0.8, random_state=42).
- Report: RMSE, MAE, R² on test set.
- Plot feature importances — top 15 features.

5.3 Cross-Validation (on best model so far)

- Apply 5-Fold cross_val_score on your best-performing model using neg_root_mean_squared_error.
- Print mean CV RMSE ± std deviation.
- Does the CV RMSE align with the test set RMSE? Discuss in 2–3 sentences.

5.4 Hyperparameter Tuning

- Use RandomizedSearchCV on Random Forest or XGBoost (at least 2 hyperparameters, n_iter=20, cv=3).
- Report best parameters and best CV RMSE.
- Re-evaluate the tuned model on the test set.

📈 Step 6: Model Evaluation & Comparison

Create a consolidated comparison table of all 5 models (Linear Regression, Ridge, Lasso, Random Forest, XGBoost) showing:

- RMSE (on original USD scale after expm1)
- MAE (on original USD scale after expm1)
- R^2 Score
- 5-Fold CV RMSE (where applicable)
- Training Time (seconds)

Present this as a formatted pandas DataFrame or table.

- Plot: Actual vs Predicted scatter for your best model — annotate RMSE and R^2 on the plot.
- Identify the best model and justify your choice in 3–5 sentences. Consider performance, interpretability, and production suitability.

💡 Real-world context: A startup like NoBroker would care about RMSE in absolute USD terms (equivalent to INR in Indian context). A model with RMSE of ₹5 lakh is very different from one with RMSE of ₹25 lakh for a homebuyer!

🔍 Step 7: Residual Analysis & Model Interpretation

7.1 Residual Plots

- Plot Residuals vs Fitted values for your best model. Are residuals randomly scattered (good) or show a pattern (bad)?
- Plot a histogram of residuals. Are they approximately normally distributed?
- Plot a Q-Q plot of residuals. Comment on any heavy tails.

7.2 Business Interpretation

- List the top 5 most impactful features from your best model.
- Write 3–5 bullet points explaining each feature in plain English. E.g.: 'OverallQual is the strongest predictor — a home rated Excellent (10) commands significantly higher prices than one rated Average (5).'
- (Optional — Bonus) Use SHAP values on your XGBoost model: plot a SHAP beeswarm summary plot and a SHAP waterfall plot for one individual prediction.

🚀 Step 8: Pipeline, Deployment & GitHub Submission

8.1 Save the Final Pipeline

- Wrap your best model + preprocessing into a single sklearn Pipeline.
- Save: joblib.dump(pipeline, 'house_price_model.pkl').
- Test it: load the pickle and predict the sale price for 5 sample rows. Print predicted vs actual prices.

8.2 GitHub Submission

- Create a GitHub repository named: house-price-regression-supervised-learning.
- Push the following:
  - HousePrice_SupervisedLearning.ipynb (fully executed notebook)
  - house_price_model.pkl (saved pipeline)
  - summary_report.md (see below)
  - requirements.txt (all libraries used)
  - README.md (project overview, dataset link, how to run)

8.3 Summary Report (summary_report.md)

Write a short report (~400–500 words) covering:

- What was the business problem and dataset?
- What preprocessing and feature engineering steps did you apply and why?
- Which model performed best and why?
- What were the top 3 most predictive features? What do they suggest to a real estate professional?
- What would you do next to improve this model in production (e.g., more data, ensembling, Bayesian optimisation)?

Marking Scheme (50 Marks)

Total Marks: 50 — distributed across 3 components as follows:

Component	Marks	Deliverable
A. Technical Completion	30 Marks	Completed .ipynb & .pkl file with all tasks built and functional
B. Recorded Video (Face + Screen)	15 Marks	5-10 min MP4/MOV showing face + screen with verbal explanation
C. GitHub Repository + README.md	5 Marks	Public GitHub repo with screenshots, README, video link, commits
TOTAL	50 Marks	

Component B: Recorded Video - Face + Screen (15 Marks)

Record a video showing your face (webcam picture-in-picture) and your computer's entire screen simultaneously. Explain concepts verbally throughout — do not just click without speaking.

Video Requirements:

Requirement	Detail
Format	MP4 or MOV, maximum 500 MB
Face visibility	Webcam picture-in-picture overlay visible throughout the full recording
Screen content	Entire screen (by demonstrating ipynb and pkl file)
Duration	5 to 10 minutes
Naming convention	Practical_Exam_YourName_GRID.mp4
Upload location	Google Drive (Anyone with link can view) or YouTube (Unlisted)
Link placement	Paste the video URL in your GitHub README.md under the Video section

Component C: GitHub Repository + README.md (5 Marks)

Create a public GitHub repository specifically for this project. Add the following files as mentioned in the Deliverables section below.

📄 Deliverables

Notebook	HousePrice_SupervisedLearning.ipynb (fully executed, all outputs visible)

Saved Model	house_price_model.pkl (sklearn Pipeline)
Summary Report	summary_report.md (theory + observations, ~1 page)
GitHub Repo	All of the above + requirements.txt + README.md

✅ **Submission Checklist:**

- ☐ Notebook runs top-to-bottom without errors
- ☐ All plots are visible in the notebook output
- ☐ Model comparison table is present (Step 6)
- ☐ Metrics are reported in original USD/INR scale (not log scale)
- ☐ GitHub repo link shared with examiner
- ☐ README.md explains how to run the project

BRING ON YOUR CODING ATTITUDE

Practical Exam — Supervised Learning